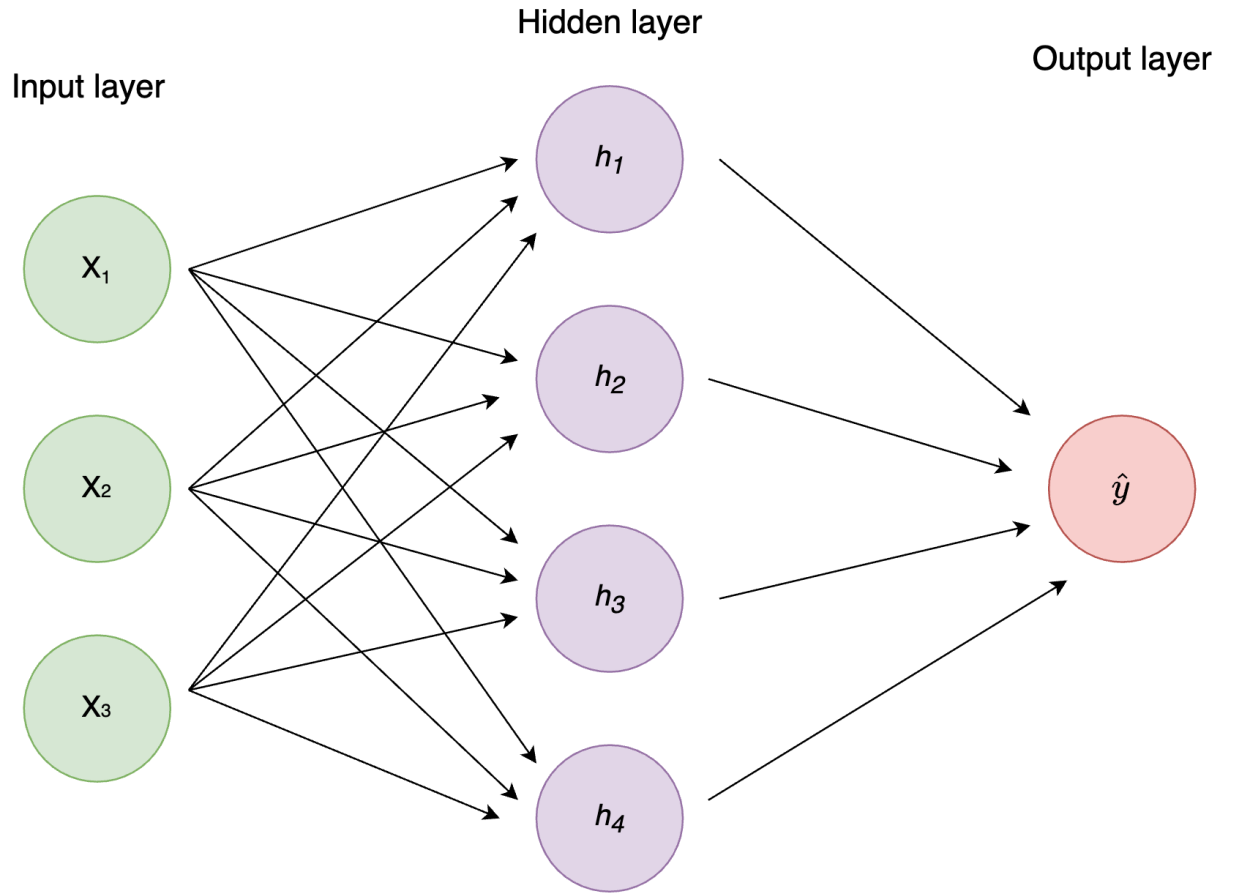


Intro to AI for Medicine

Lec 3: Loss Functions

Last Time

- Learned about matrices
- Defined the neural net
- Learned how we use the neural net to update matrices



Agenda

- Define loss function
- Linear regression
 - How we change that to multidimensional regression of any function
 - Mean Squared Error Loss
- Classification Intro
 - Log Loss
- Classification Case Study
 - ML model evaluation metrics (mostly review)
- Multi-Class Classification

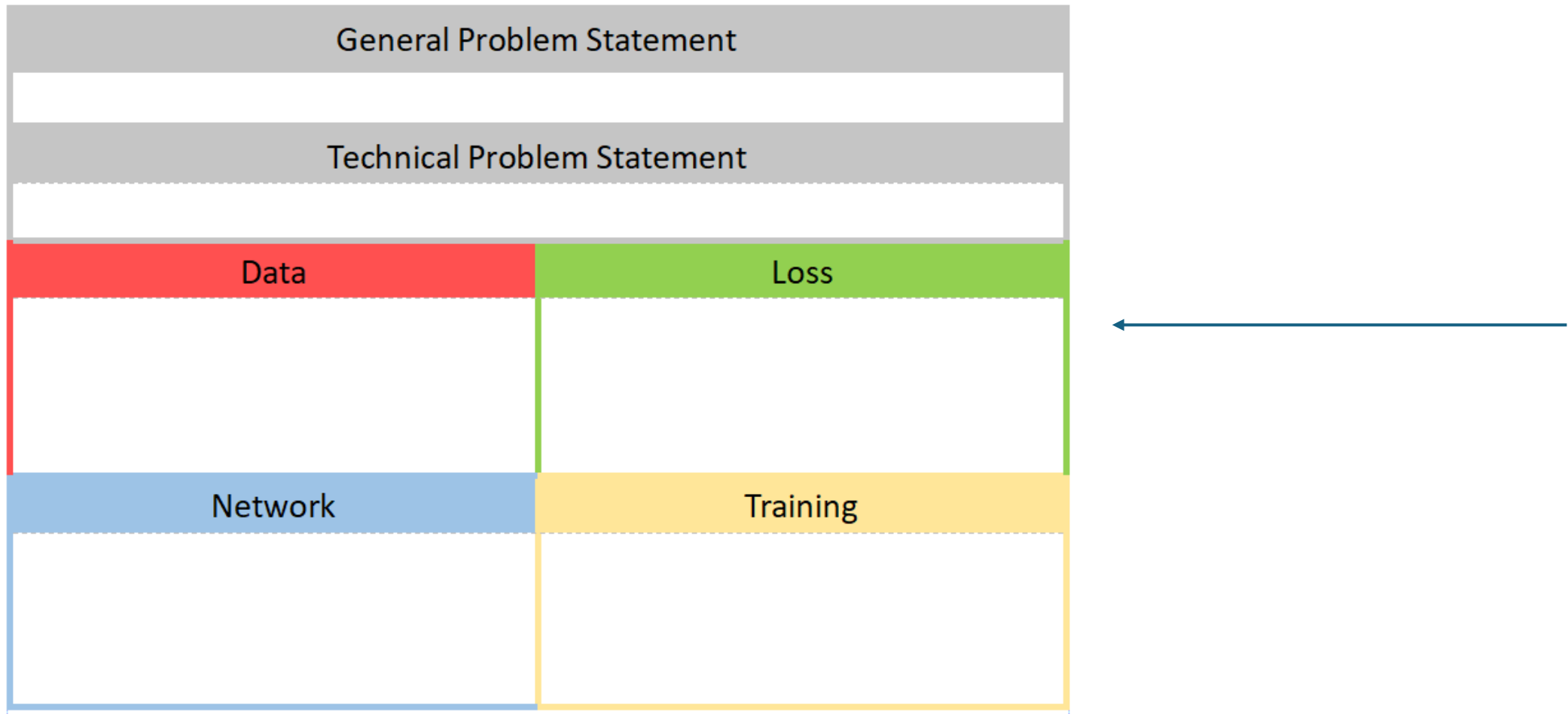
Loss Functions

- How do we grade our results?
- Use a loss function!
- Loss function: Definition of correctness, which when applied over the data, decreases when the hypotheses are more correct
- Training error is the sum of loss across all data points.

$$\mathcal{E}_n(h) = \frac{1}{n} \sum_{i=1}^n \mathcal{L}(h(x^{(i)}), y^{(i)})$$

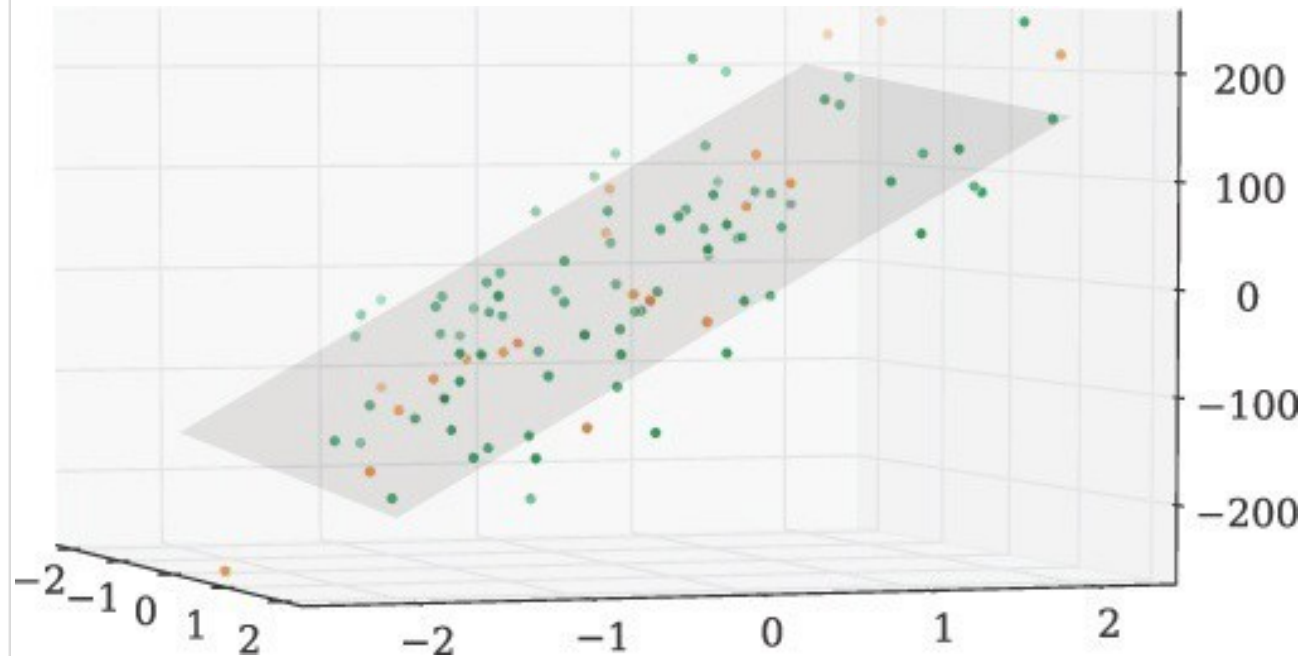
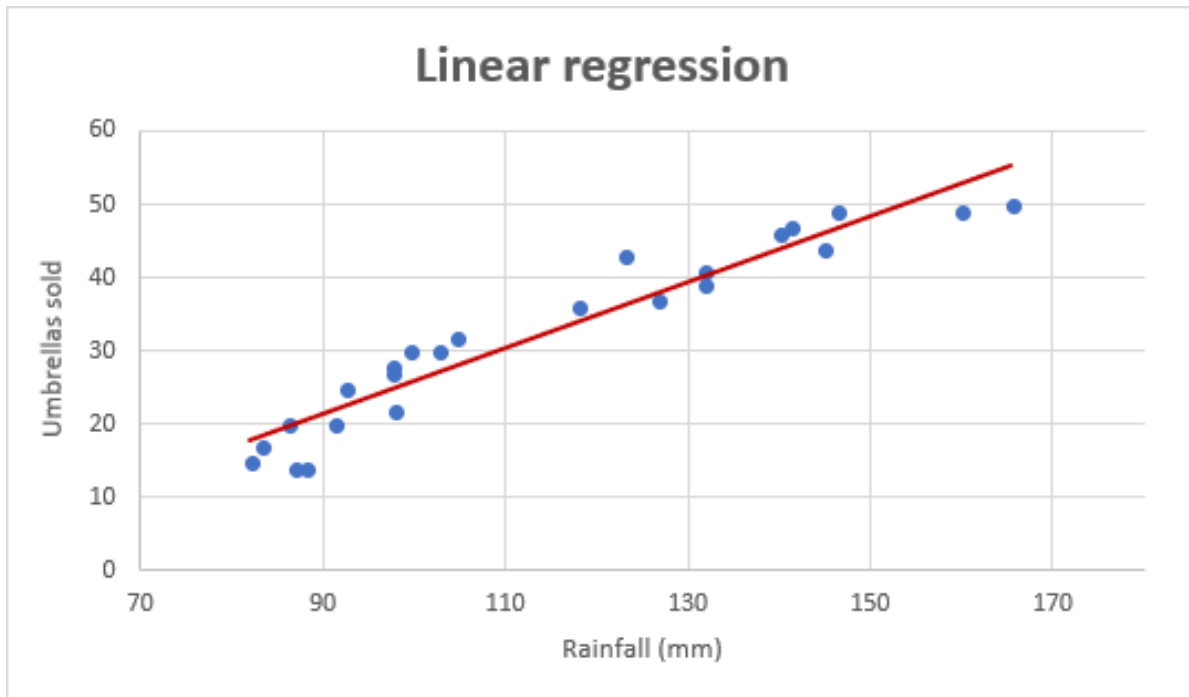
This time

- We learn how to define success via loss functions



What is Linear Regression?

- Find a linear function that best models the relationship between inputs and outputs



Linear Regression: Problem

- Given an input matrix x , an output matrix y , use x to predict y .

Goal

$$[x_1 \ x_2 \ \dots \ x_n] \rightarrow [y_1 \ y_2 \ \dots \ y_n]$$

Plan

$$[x_1 \ x_2 \ \dots \ x_n] \rightarrow [\hat{y}_1 \ \hat{y}_2 \ \dots \ \hat{y}_n]$$

$$x \rightarrow \boxed{h} \rightarrow y$$

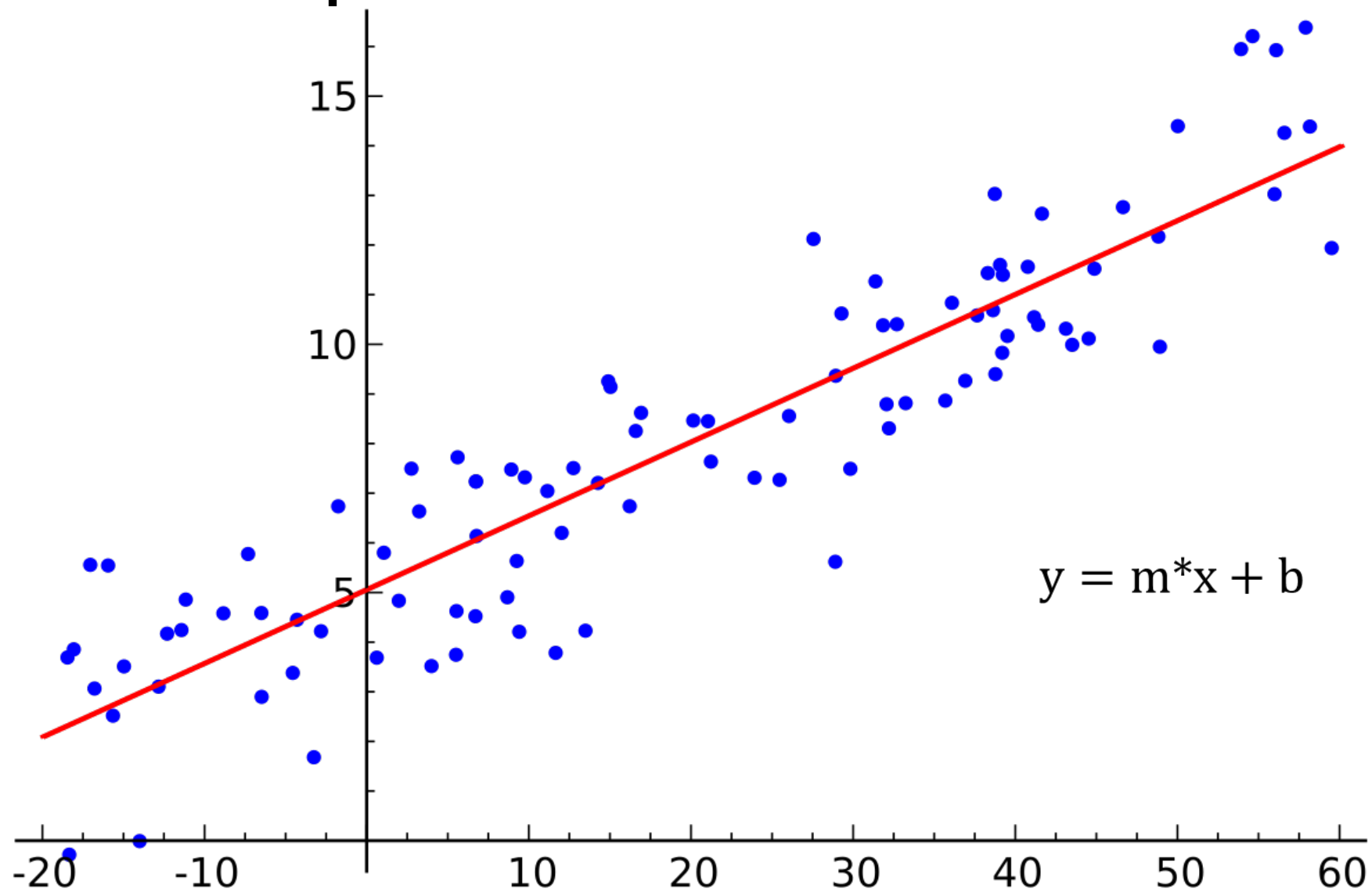
Linear Regression: Plan

1. Take in an input
2. Multiply it by a constant weight
 1. Or each part of the input by its own weight, if input has multiple parts/dimensions
3. See how we did
4. Update our weights
5. Repeat

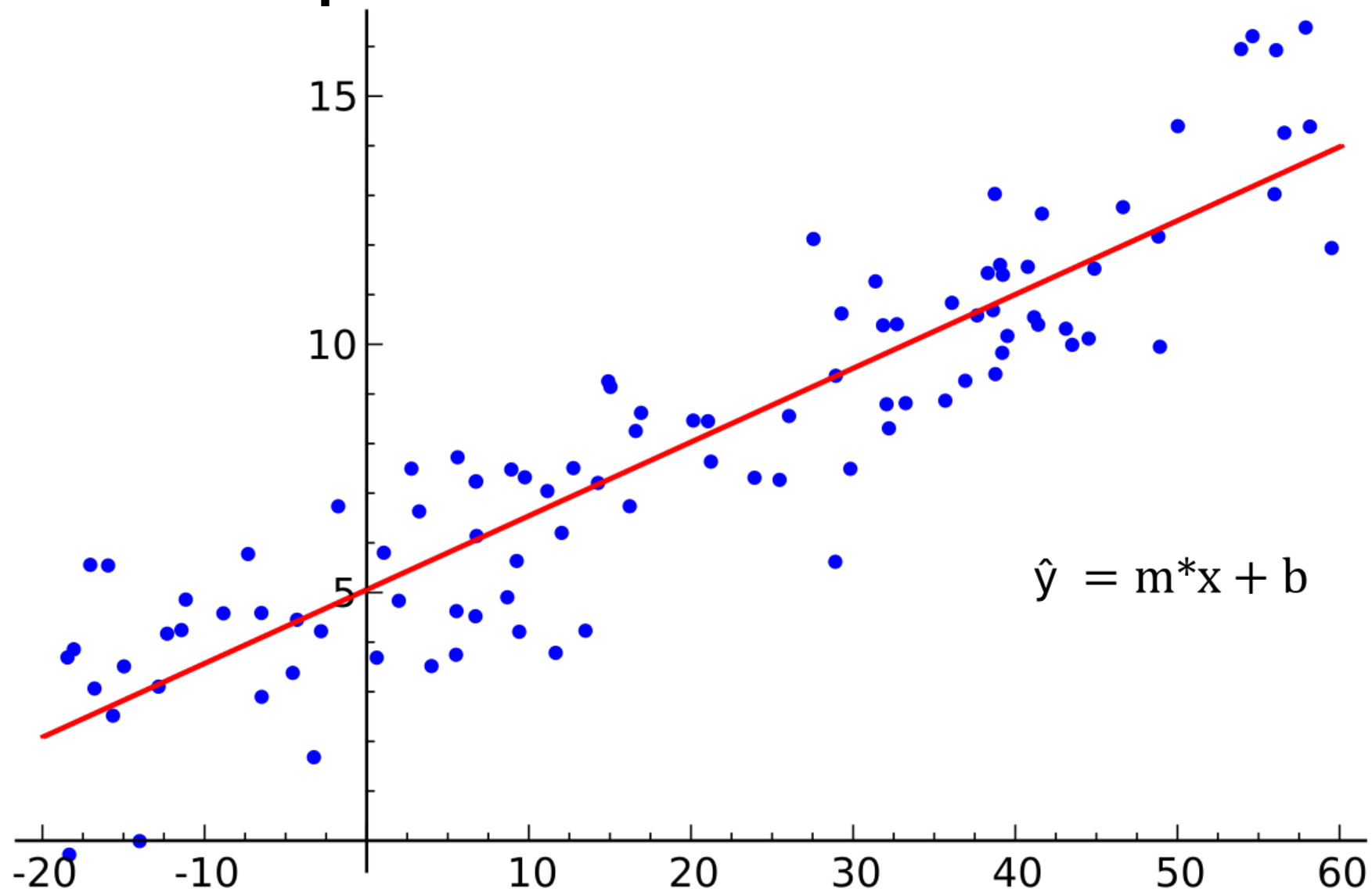
Loss Types

Loss type	Definition	Equation
L₁ loss	The sum of the absolute values of the difference between the predicted values and the actual values.	$\sum actual\ value - predicted\ value $
Mean absolute error (MAE)	The average of L ₁ losses across a set of *N* examples.	$\frac{1}{N} \sum actual\ value - predicted\ value $
L₂ loss	The sum of the squared difference between the predicted values and the actual values.	$\sum (actual\ value - predicted\ value)^2$
Mean squared error (MSE)	The average of L ₂ losses across a set of *N* examples.	$\frac{1}{N} \sum (actual\ value - predicted\ value)^2$

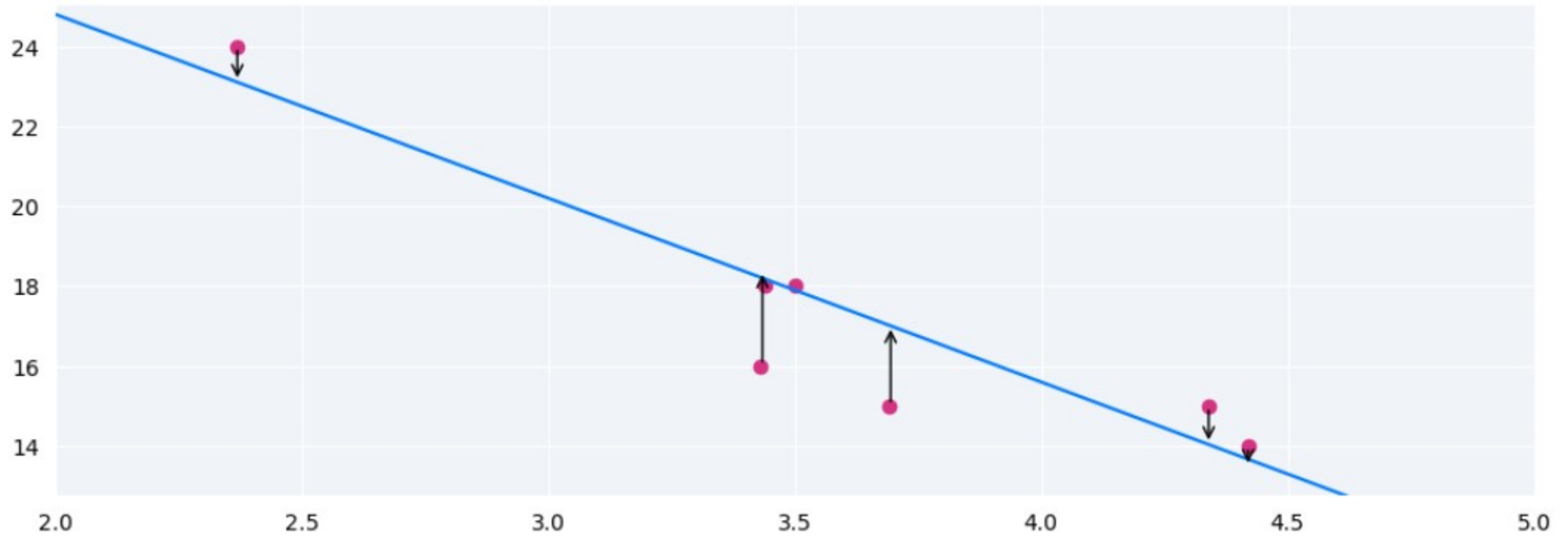
1D Input Case



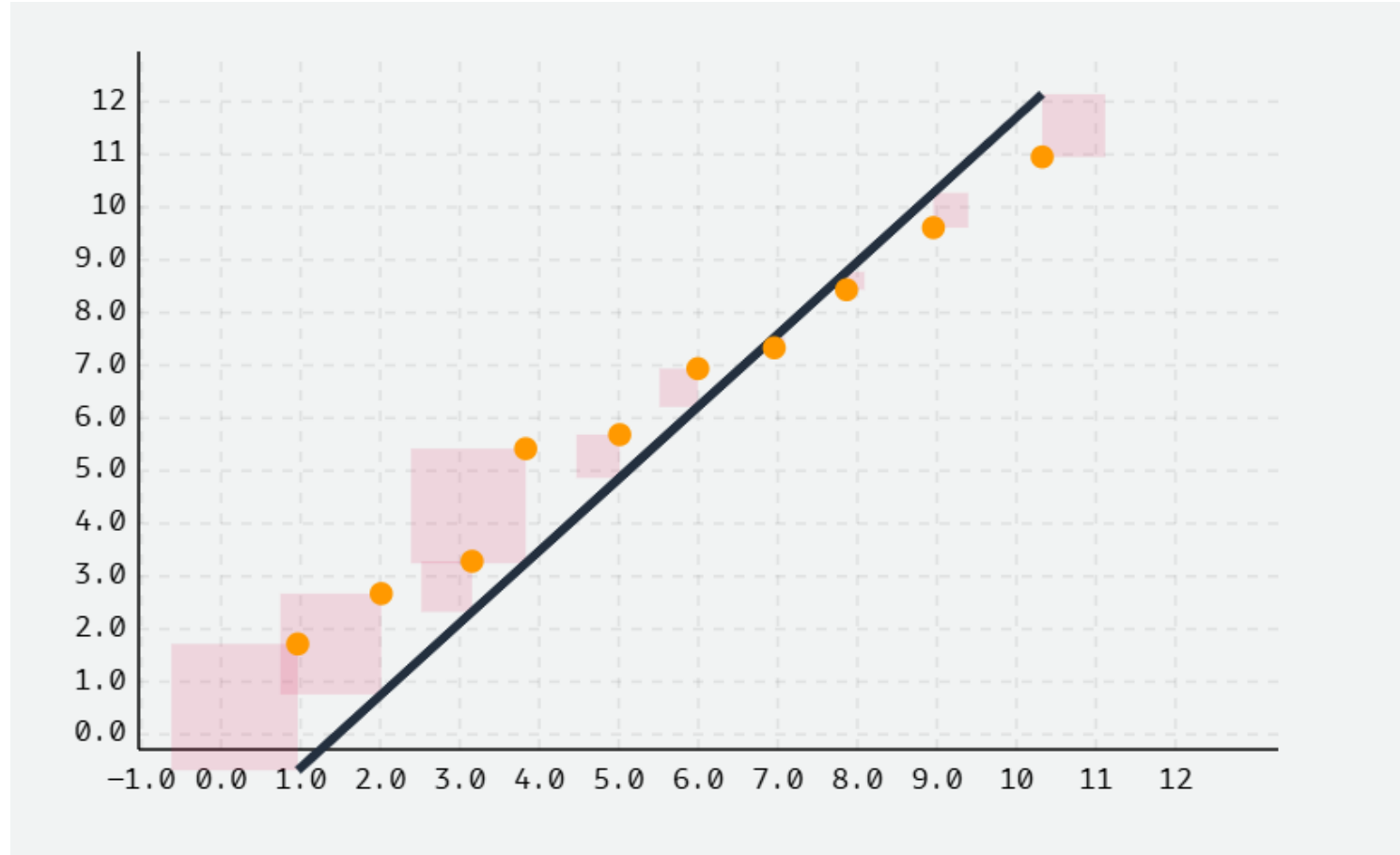
1D input Case



Loss Functions: Visualized



Loss Functions: Visualized



Linear Regression: Problem

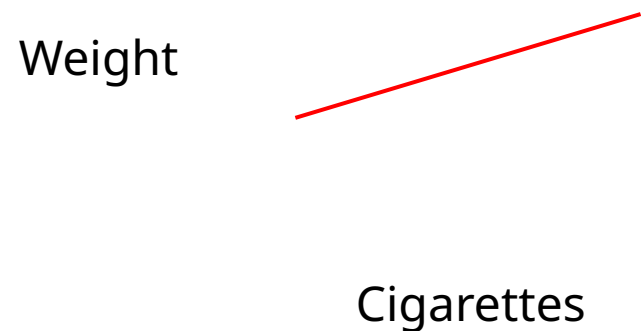
InfantID	Cigarettes_per_day (during pregnancy)	BirthWeight (grams)
001	0	3500
002	5	3300
003	10	3150
004	0	3600
005	15	3000
006	20	2850
007	0	3400
008	25	2750
009	12	3100
010	0	3550

Linear Regression: Intuition

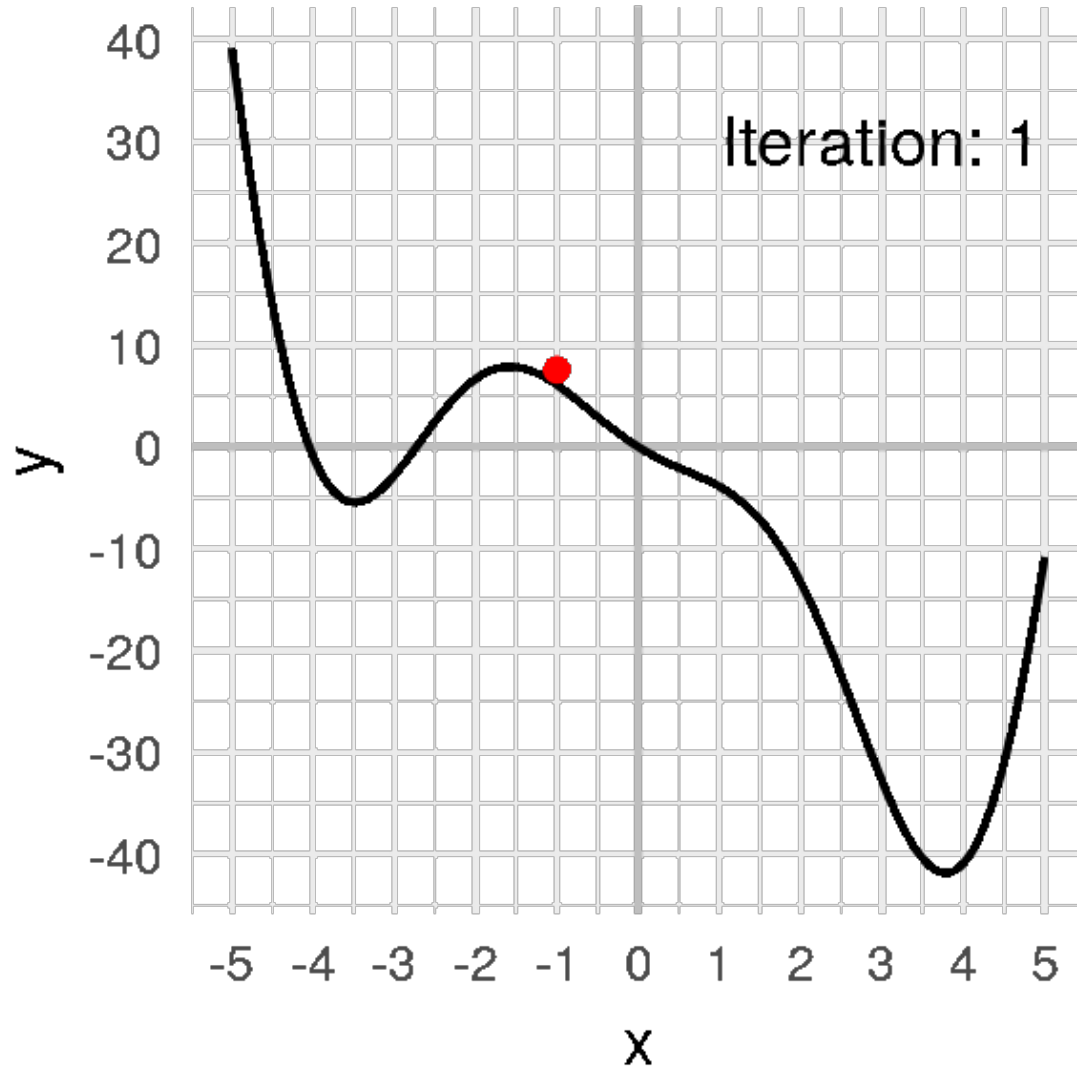
InfantID	Cigarettes_per_day (during pregnancy)	BirthWeight (grams)	Prediction \$	Difference	Squared Error
001	0	3500	3000	500	250000
002	5	3300	3250	50	2500
003	10	3150	3500	-350	122500
					385000

$$\text{Weight} = 250 * \text{cigarettes} + 3000$$

- Imagine if our prediction was exact – error contribution for that point is 0
- So, lower is better which matches gradient descent objective.



Gradient Descent



Algorithm:

1. Find gradient at start point
2. Move opposite that direction by an amount = gradient magnitude times a constant (learning rate)
3. Repeat until the value between steps changes less than a set amount (or a fixed number of times)

$$\mathbf{a}_{n+1} = \mathbf{a}_n - \eta \nabla f(\mathbf{a}_n)$$

Linear Regression: Intuition

InfantID	Cigarettes_per_day (during pregnancy)	BirthWeight (grams)	Predictions	Difference	Squared Error
001	0	3500	3250	250	62500
002	5	3300	3250	50	2500
003	10	3150	3250	-350	10000
					75000

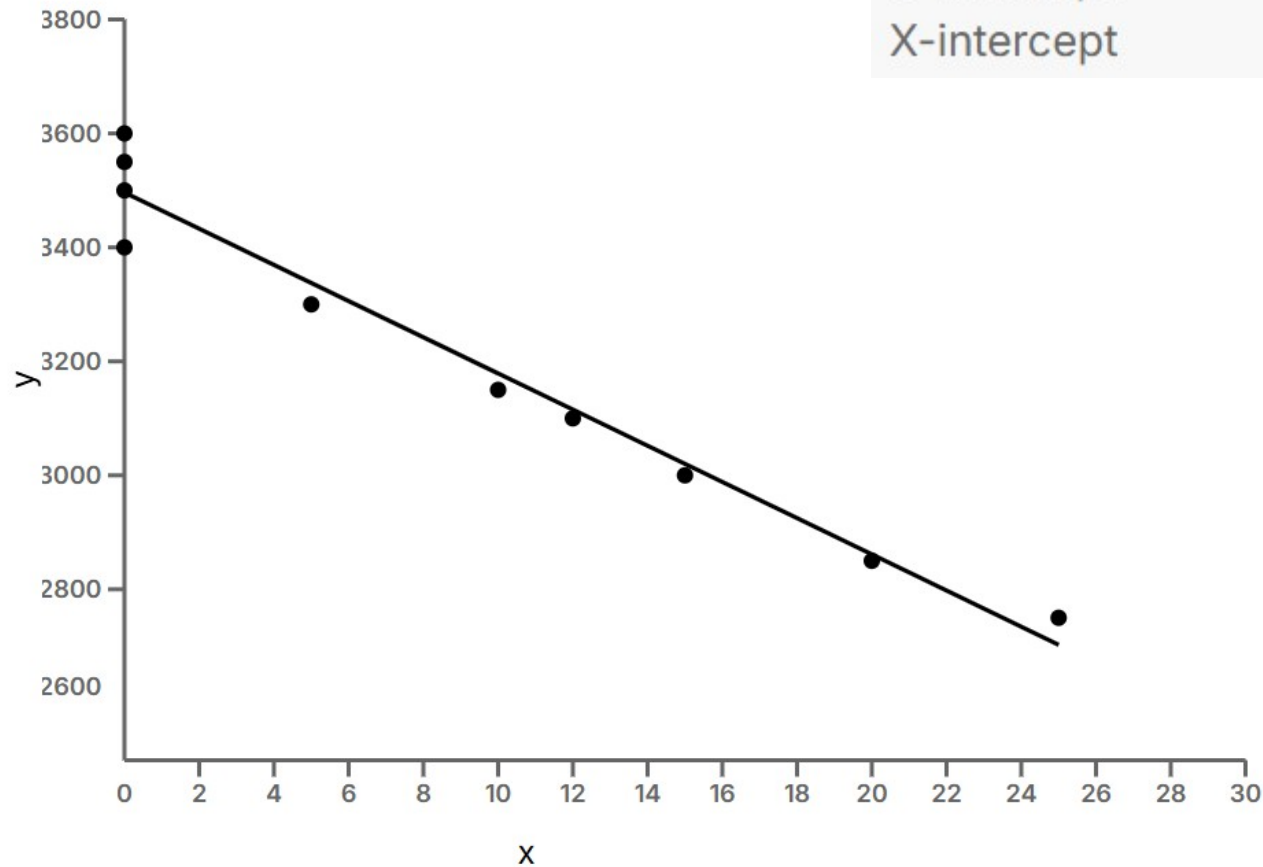
$\text{Weight} = 0 * \text{cigarettes} + 3000$



Linear Regression

Best-fit values

Slope	-31.74 ± 2.150
Y-intercept	3496 ± 26.50
X-intercept	110.1



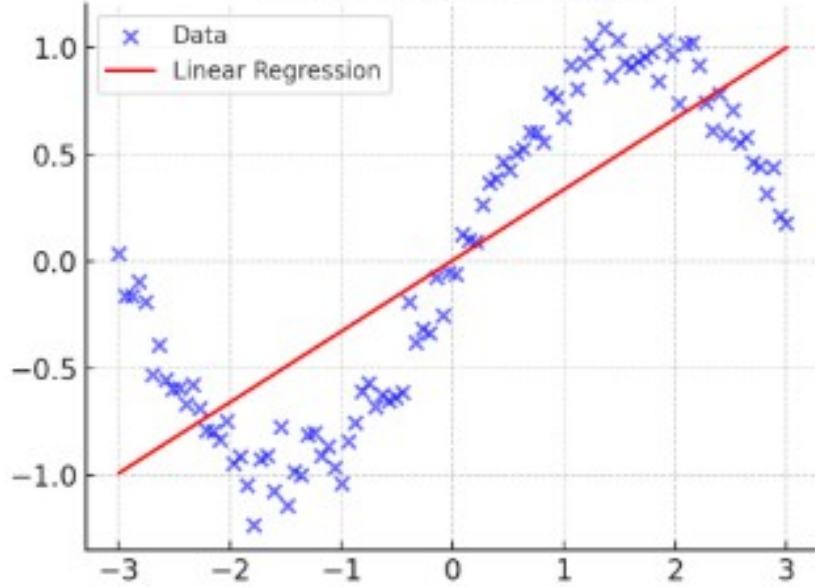
$$\begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} * [w] = \begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \vdots \\ \hat{y}_n \end{bmatrix}$$

Regression Summary

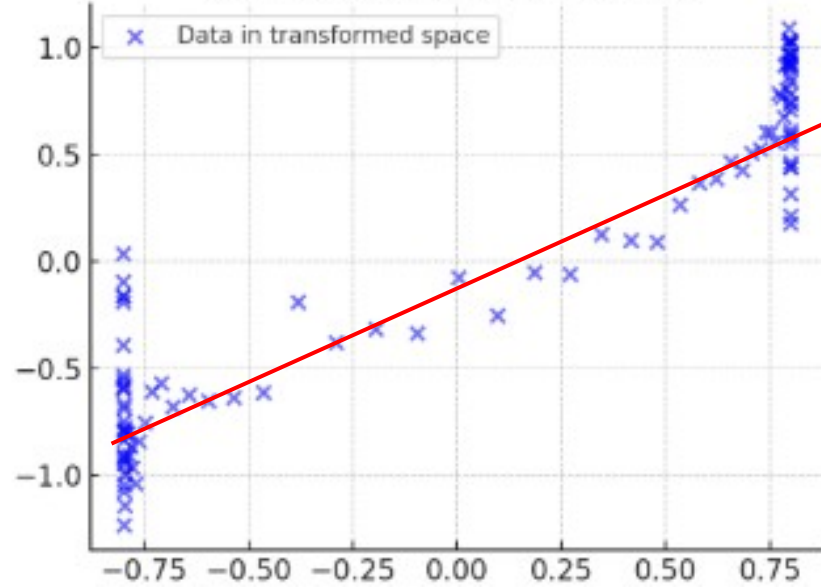
- Solvable using ML
- Mean squared error loss makes most sense for these problems
 - Numerical input and output
 - Big outliers are bad

What if It's Nonlinear?

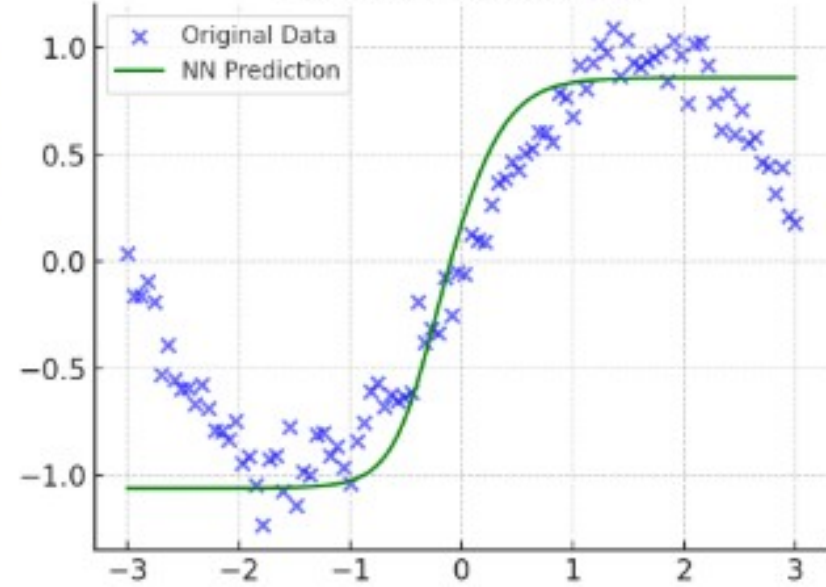
Linear Regression Fit



After Nonlinear Activation



Neural Network Fit

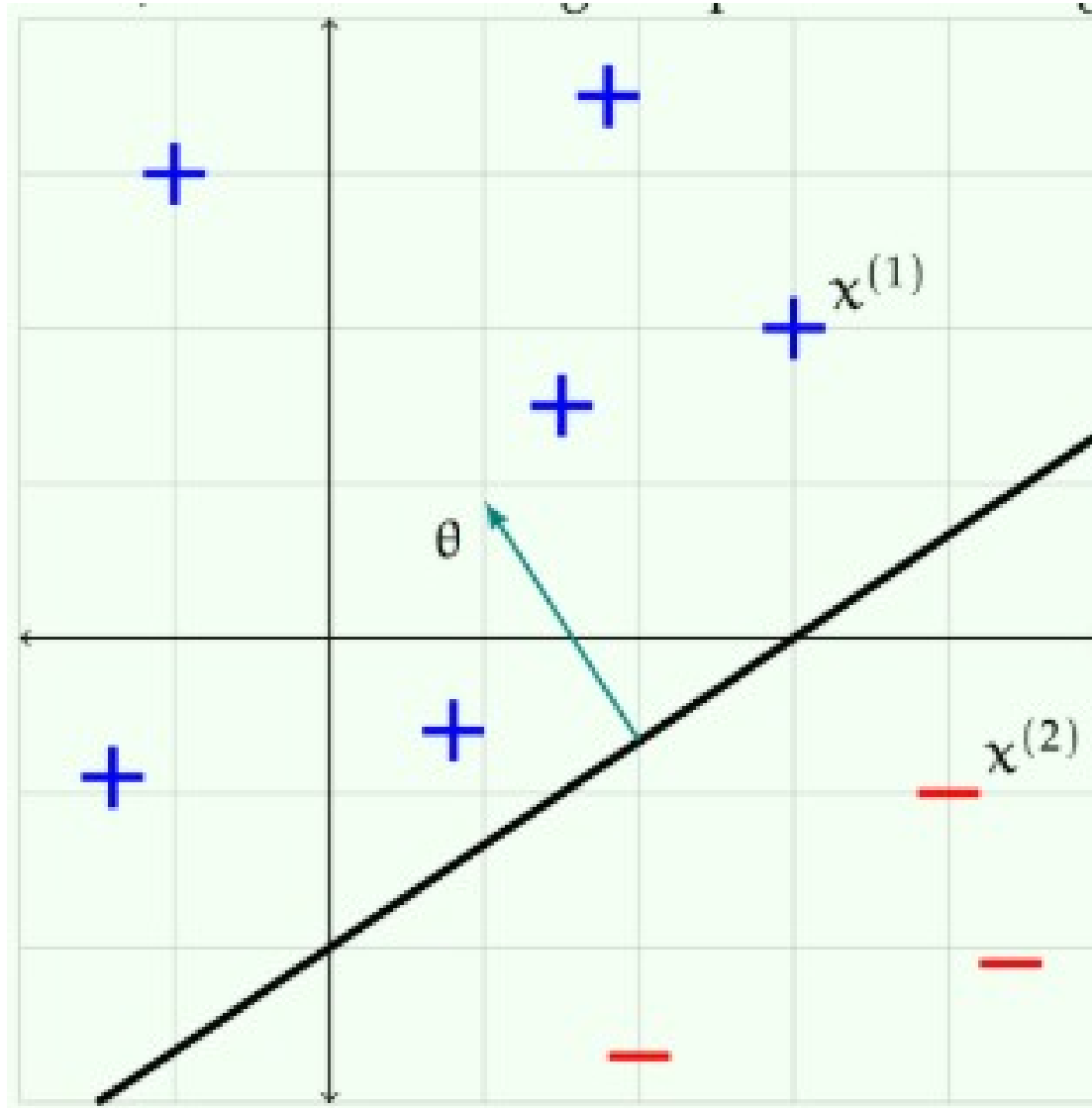


A New Problem Statement

- Let's say we want to predict probability a patient has a disease.
- We want to “classify” them into one of two categories.
- So instead of a regression problem, we have a classification problem.

$$h(x; \theta, \theta_0) = \text{sign}(\theta^T x + \theta_0) = \begin{cases} +1 & \text{if } \theta^T x + \theta_0 > 0 \\ -1 & \text{otherwise} \end{cases}.$$

What Loss Function?



Binary Loss:

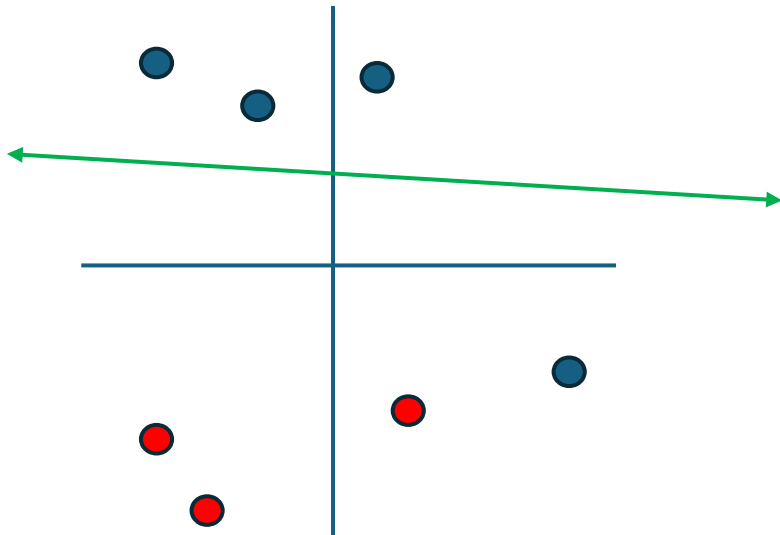
If on the correct side, 0

If on the wrong side, 1

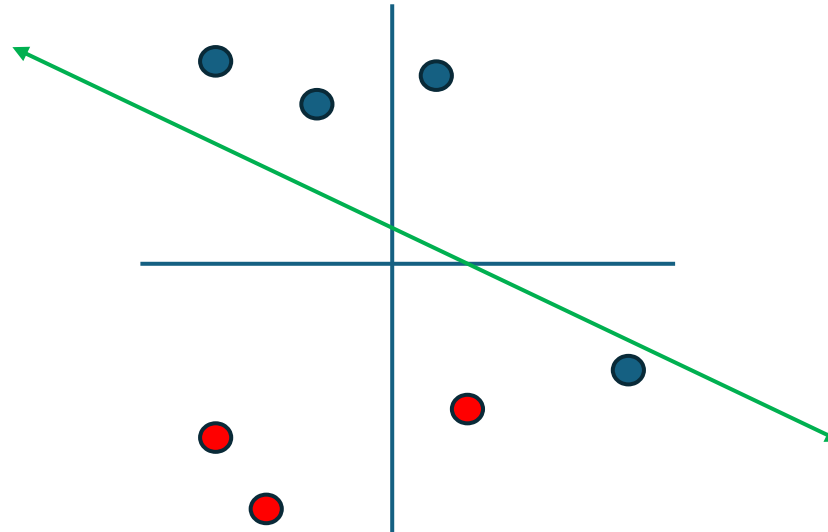
Binary Loss:

Binary Loss:
If on the correct side, 0
If on the wrong side, 1

Loss = 1



Loss = 1

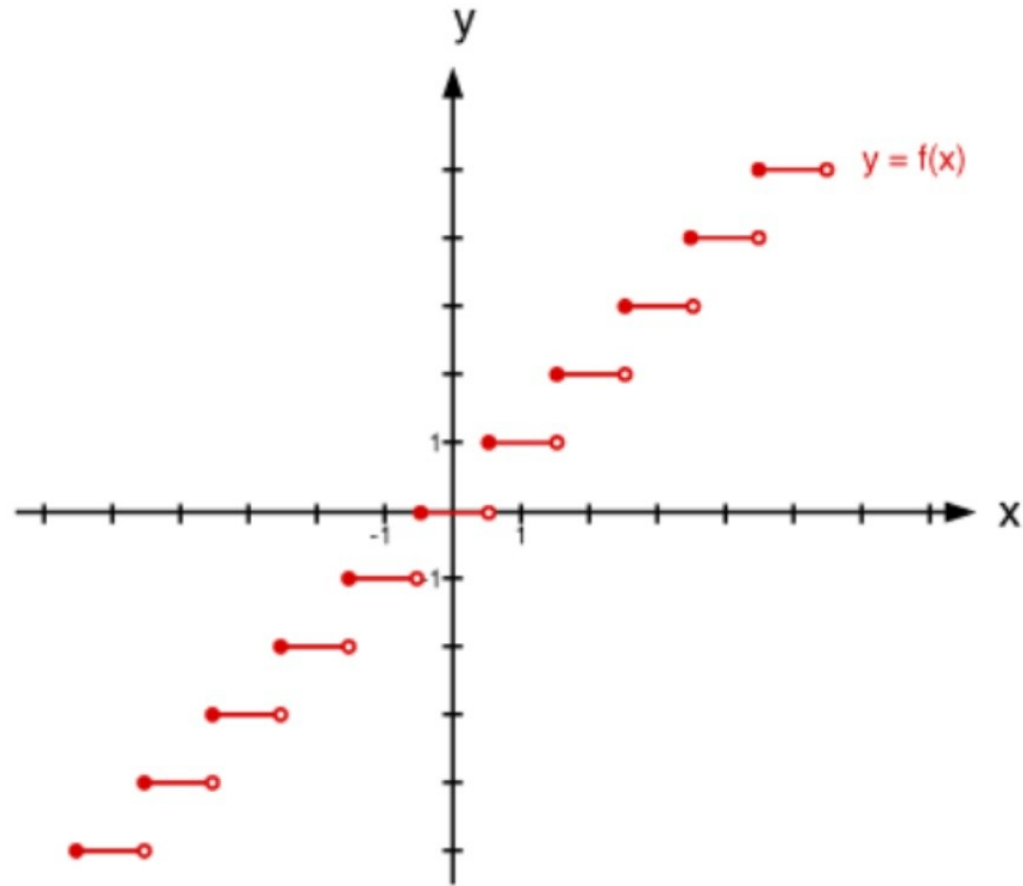


We already know this!

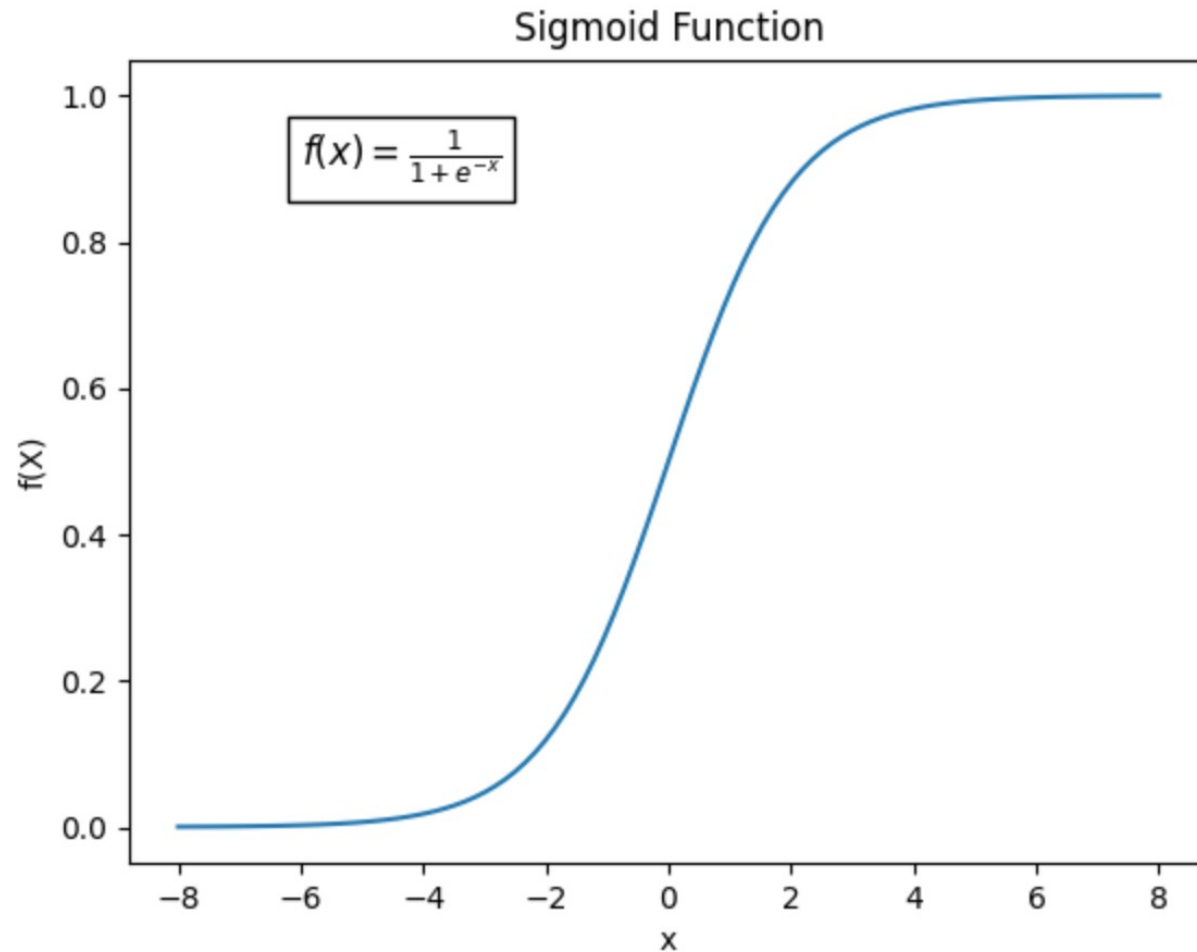
Confusion matrix

	Actual positive	Actual negative
Predicted positive	True positive (TP): A spam email correctly classified as a spam email. These are the spam messages automatically sent to the spam folder.	False positive (FP): A not-spam email misclassified as spam. These are the legitimate emails that wind up in the spam folder.
Predicted negative	False negative (FN): A spam email misclassified as not-spam. These are spam emails that aren't caught by the spam filter and make their way into the inbox.	True negative (TN): A not-spam email correctly classified as not-spam. These are the legitimate emails that are sent directly to the inbox.

Binary Loss Surface



Sigmoid and Log Loss



$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

$$h(x; \theta, \theta_0) = \sigma(\theta^T x + \theta_0)$$

Log Likelihood Loss

$$\mathcal{L}_{\text{nll}}(\text{guess}, \text{actual}) = -(\text{actual} \cdot \log(\text{guess}) + (1 - \text{actual}) \cdot \log(1 - \text{guess}))$$

Recall:

$\log(0) \rightarrow \text{negative infinity}$

$\log(1) = 0$

Guess	Actual	Loss
0.00001	0	0
1	0	infinity
1	1	0
0.00001	1	infinity
.5	1	.301
.75	1	.125

Log Likelihood Loss

Note: often called cross-entropy loss!

$$\mathcal{L}_{\text{nll}}(\text{guess}, \text{actual}) = -(\text{actual} \cdot \log(\text{guess}) + (1 - \text{actual}) \cdot \log(1 - \text{guess}))$$

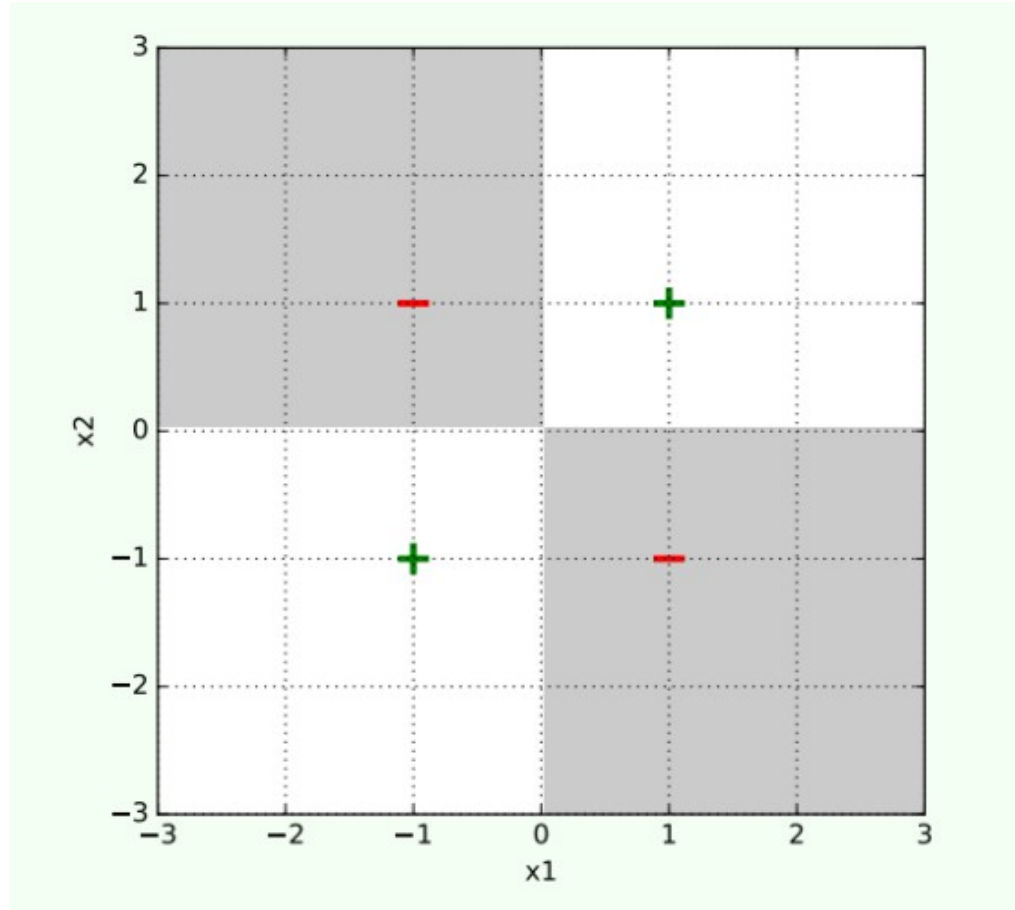
Recall:

$\log(0) \rightarrow \text{negative infinity}$

$\log(1) = 0$

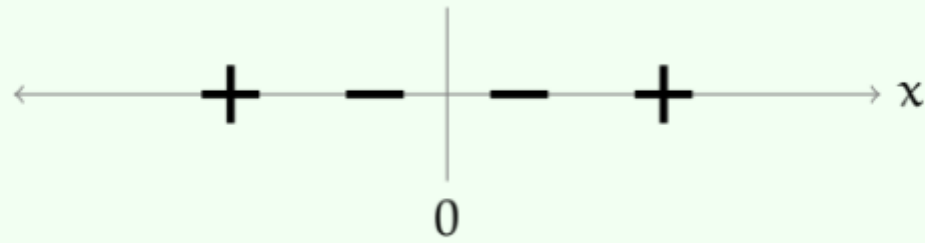
Guess	Actual	Loss
0.00001	0	0
1	0	infinity
1	1	0
0.00001	1	infinity
.5	1	.301
.75	1	.125

What if we can't separate?



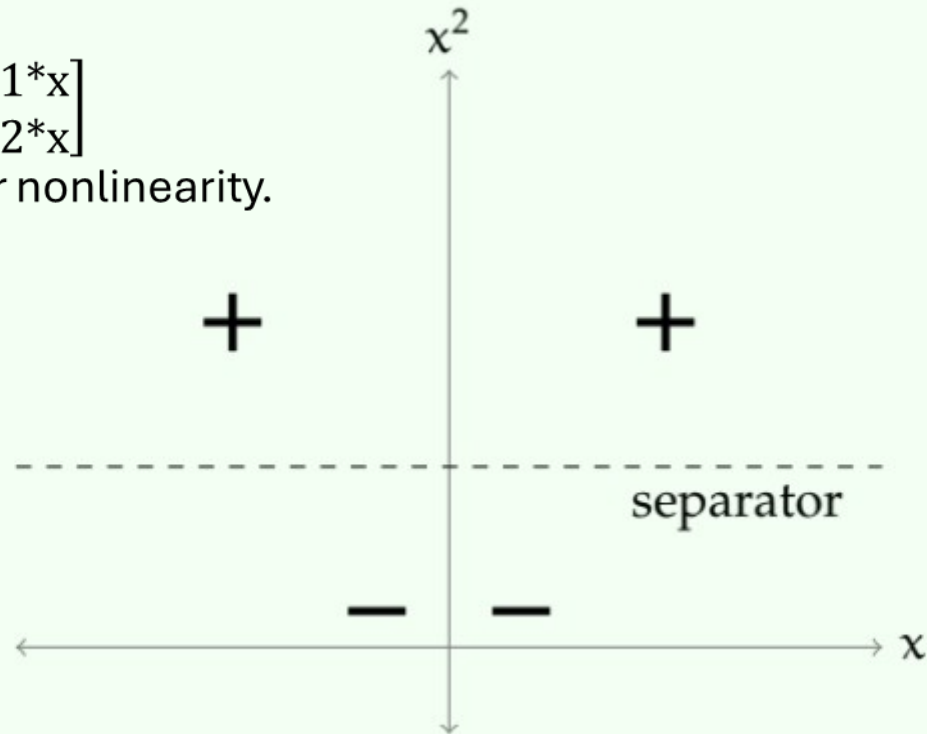
XOR

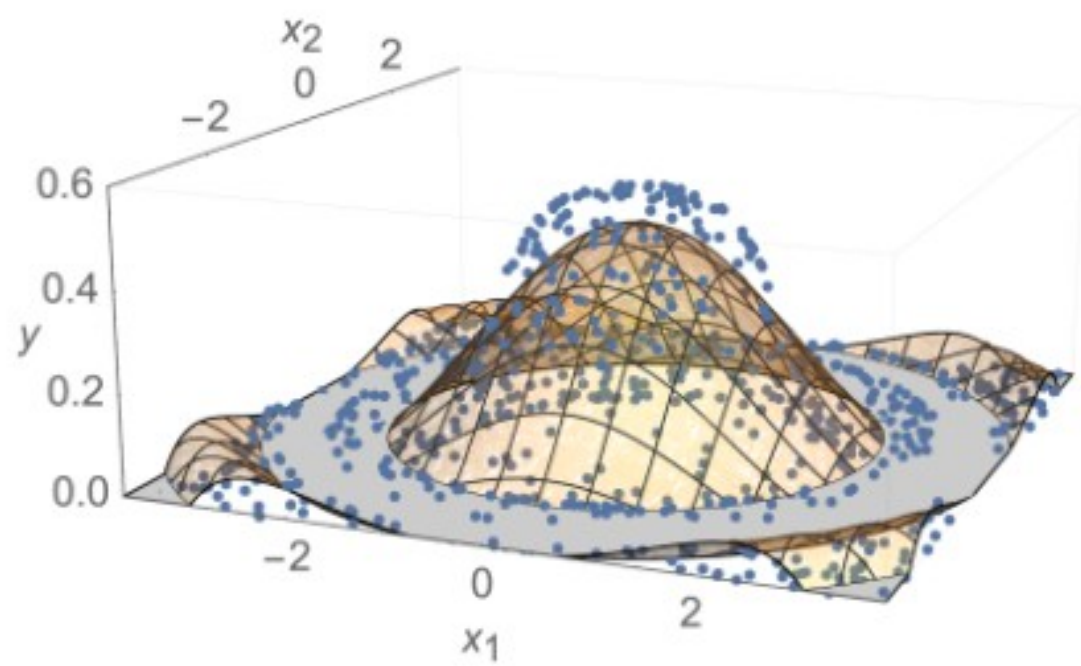
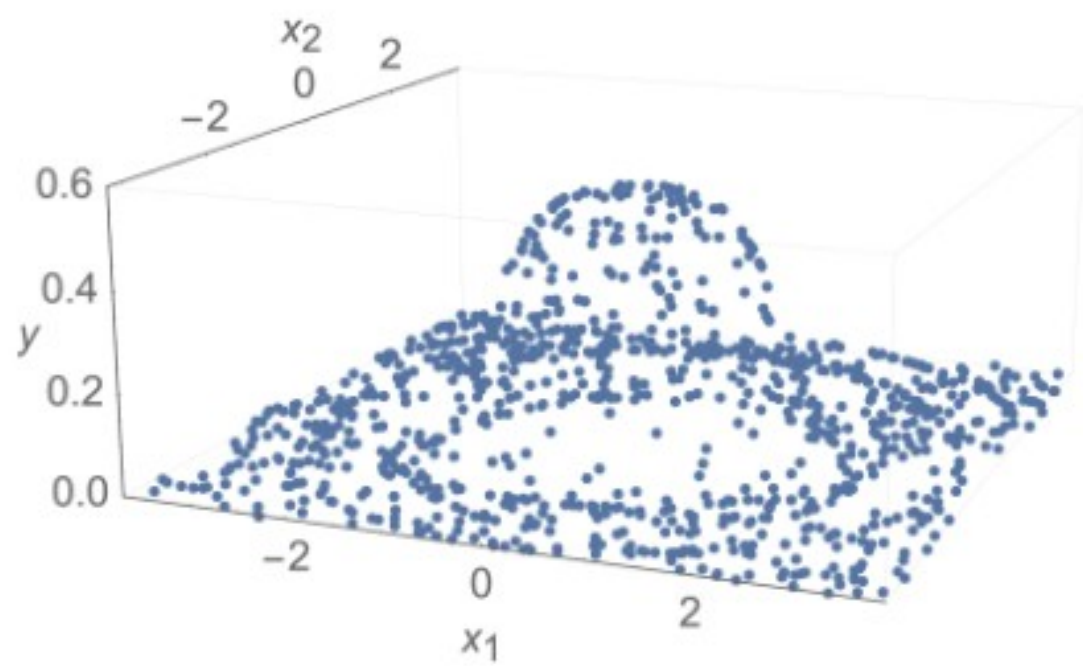
Input		Output
A	B	Q
0	0	0
0	1	1
1	0	1
1	1	0

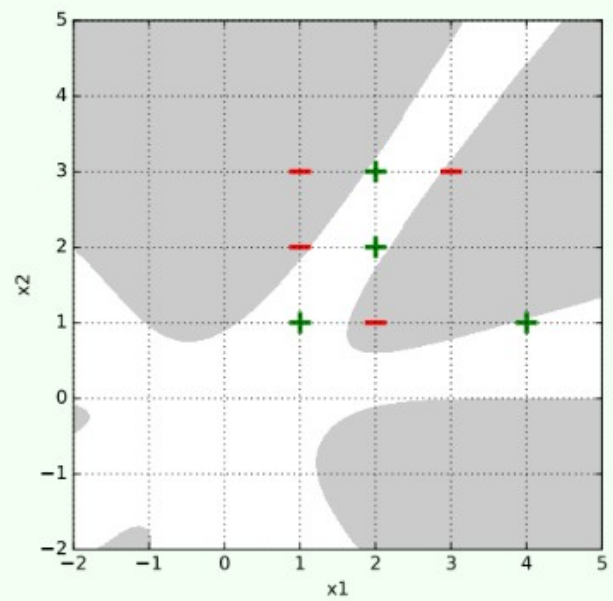
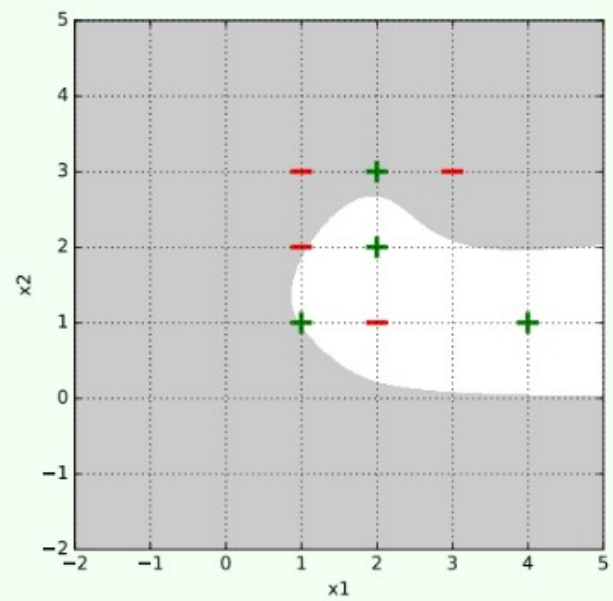
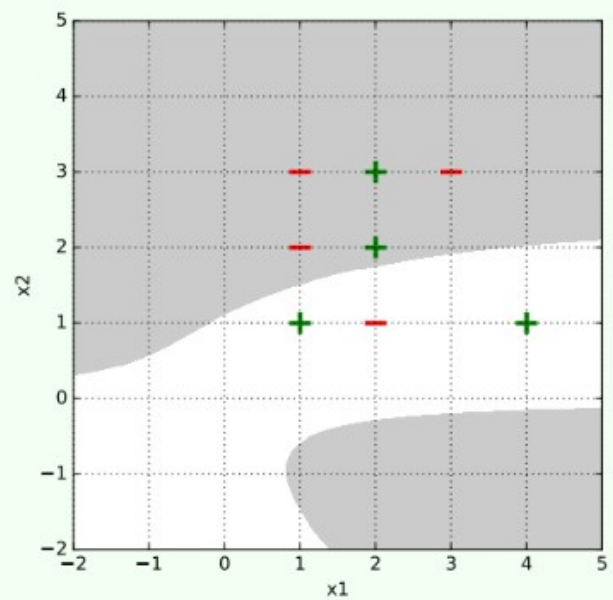
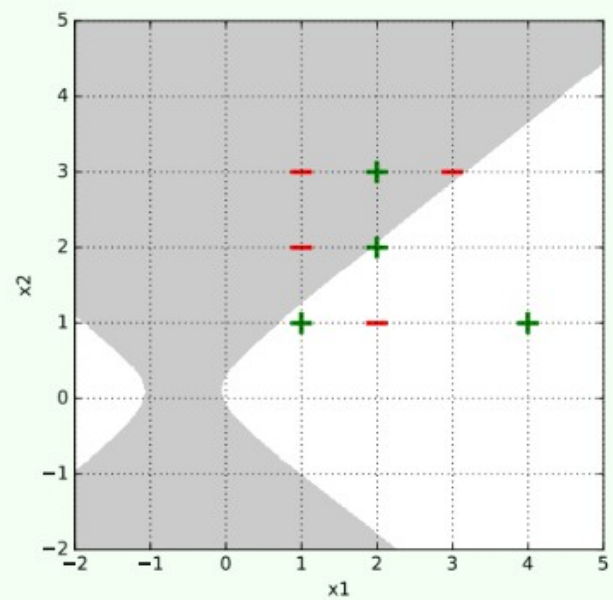


$$\begin{bmatrix} w1 \\ w2 \end{bmatrix} [x] = \begin{bmatrix} w1 * x \\ w2 * x^2 \end{bmatrix}$$
 Then with activation for nonlinearity.

$$\varphi(x) = [x, x^2].$$







Case Study

RESEARCH ARTICLE

Open Access



Comparison of deep learning with regression analysis in creating predictive models for SARS-CoV-2 outcomes

- Compares Deep Learning to regular statistical model (Cox Regression)
- Aims to find those patients who are positive for Covid and will die while hospitalized

General Problem Statement

Predict whether or not a Covid + patient will die while hospitalized

Technical Problem Statement

Deep Learning based binary classification into survive/dead groups

Data

Predictors listed on next slide
Lab diagnosis of COVID-19 in one SW London hospital
Extracted from EHR via chart review (retrospective)

Loss

Lowest loss as measured by binary cross-entropy on the validation set.
Appropriate for this problem

Network

"Artificial Neural Network"
Basically MLP/Fully connected network.
(regular network)

Training

Predictors

['age', 'sex', 'chronic_respiratory_disease', 'obesity',
'hypertension', 'diabetes', 'ischaemic_heart_disease',
'cardiac_failure', 'chronic_liver_disease',
'chronic_kidney_disease', 'cerebrovascular_event', 'fever',
'cough', 'dyspnoea', 'myalgia', 'abdominal_pain',
'diarrhoea_vomiting', 'confusion', 'collapse', 'olfactory_change',
'days_of_symptoms_prior_to_admission']

Evaluation Metrics

- **What are Evaluation Metrics?**
- Tools to **measure model performance** on unseen data.
- They tell us *how good the model is* at solving the real-world task.
- Different tasks → different metrics (classification, regression, survival).

Evaluation Metrics

Classification

- **Accuracy** – overall % correct predictions.
- **Precision** – how reliable are positive predictions?
- **Recall (Sensitivity)** – how many true positives did we catch?
- **F1 Score** – balances precision & recall.
- **AUROC** – distinguishes between classes across thresholds.

Evaluation Metrics: Accuracy

- The proportion of all classifications that were correct, whether positive or negative. It is mathematically defined as:

$$\text{Accuracy} = \frac{\text{correct classifications}}{\text{total classifications}} = \frac{TP + TN}{TP + TN + FP + FN}$$

- Perfect model has 0 FP or FN, so accuracy = 1.
- Crap model has 0 TP or TN, so accuracy = 0.
- Good for the general case – seeing how your model does.

Evaluation Metrics: Recall (True Positive Rate)

- Proportion of all actual positives that were classified correctly as positives, is also known as recall. It is mathematically defined as:

$$\text{Recall (or TPR)} = \frac{\text{correctly classified actual positives}}{\text{all actual positives}} = \frac{TP}{TP + FN}$$

- Ranges from 0-1, as before
- This is why another name for recall is **probability of detection**: it answers the question "What fraction of positive tests are truly positive for the disease?"

Evaluation Metrics: Precision (PPV)

$$\text{Precision} = \frac{\text{correctly classified actual positives}}{\text{everything classified as positive}} = \frac{TP}{TP + FP}$$

Evaluation Metrics: F1 Score

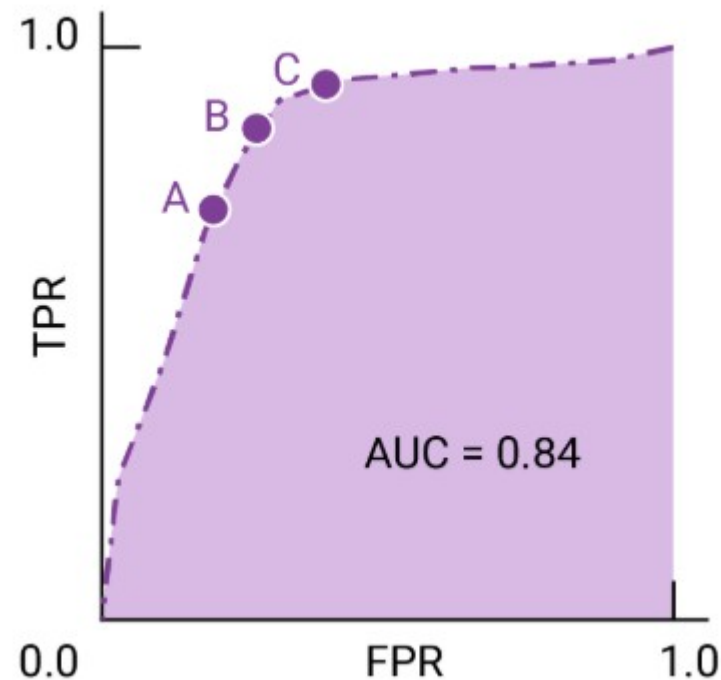
- Harmonic mean of PPV (Precision) and TPR (Recall). Defined as follows:

$$F_1 = \left(\frac{\text{recall}^{-1} + \text{precision}^{-1}}{2} \right)^{-1} = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}} = \frac{2TP}{2TP + FP + FN}$$

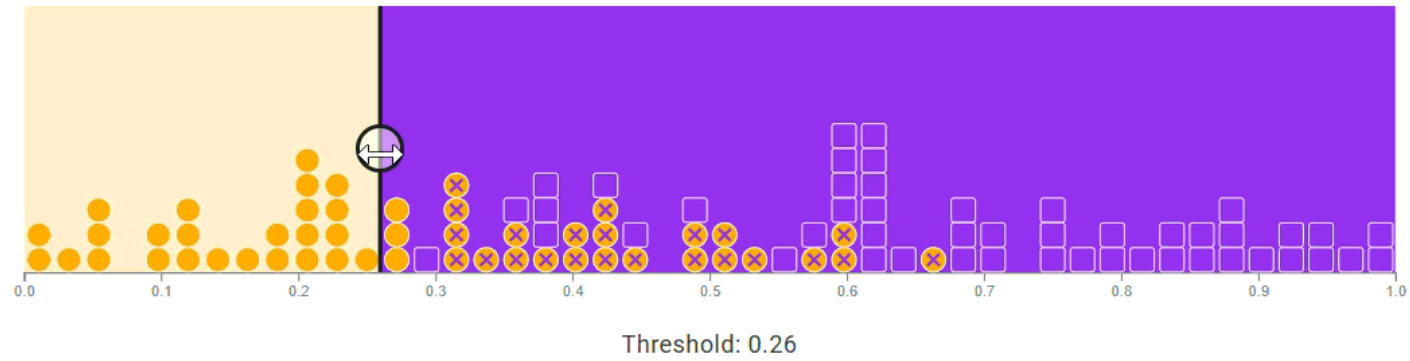
- Ranges from 0-1, as before
- Balances extremes in PPV and TPR – must not change one to cheat the other

Evaluation Metrics: Receiver-operating characteristic curve (ROC)

- Visual representation of model performance across all thresholds
- The ROC curve is drawn by calculating the true positive rate (TPR) and false positive rate (FPR) at every possible threshold



Classification threshold



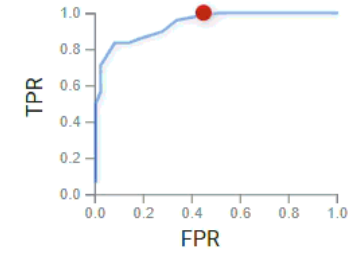
Confusion matrix

	Actually positive	Actually negative
Predicted positive	□ TP=48	⊗ FP=23
Predicted negative	⊗ FN=0	● TN=28

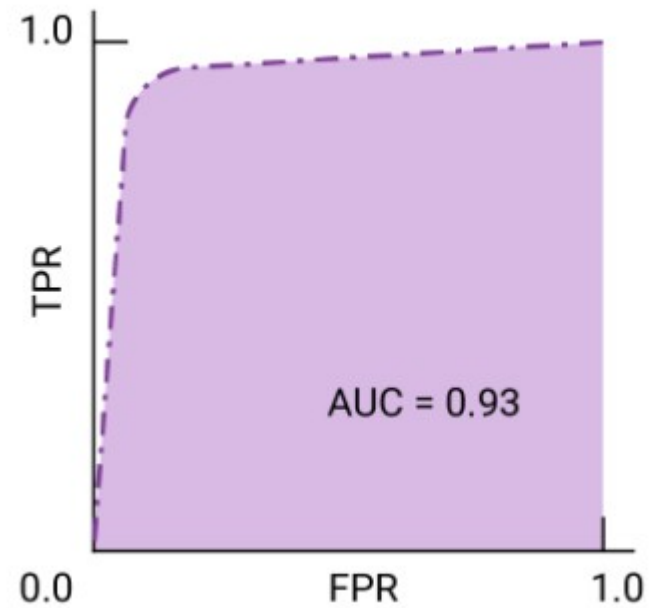
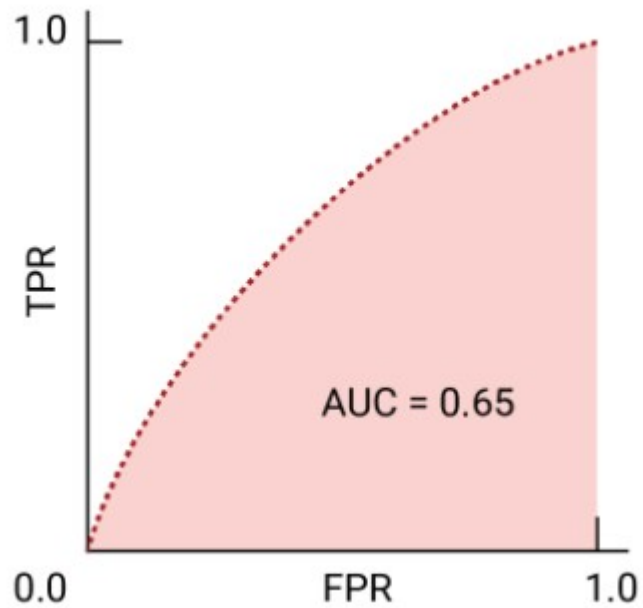
Metrics

Accuracy 0.77
 Precision 0.68
 Recall 1.00

ROC curve



Evaluation Metrics: Area Under ROC



Back to Our Example

	Cox regression model (95% CI)	ANN model (95% CI)
Sensitivity	50.0% (28.2–71.8)	64.7% (38.3–85.8)
Specificity	96.6% (88.1–99.6)	96.8% (89–99.6)
Positive predictive value	84.6% (57.0–95.8)	84.6% (57.4–95.7)
Negative predictive value	83.6% (77.0–88.6)	91.4% (84.2–95.1)
Accuracy	83.75% (73.8–91.1)	90.0% (81.2–95.6)
AUROC	86.9% (85.7–88.2)	92.6% (91.1–94.1)

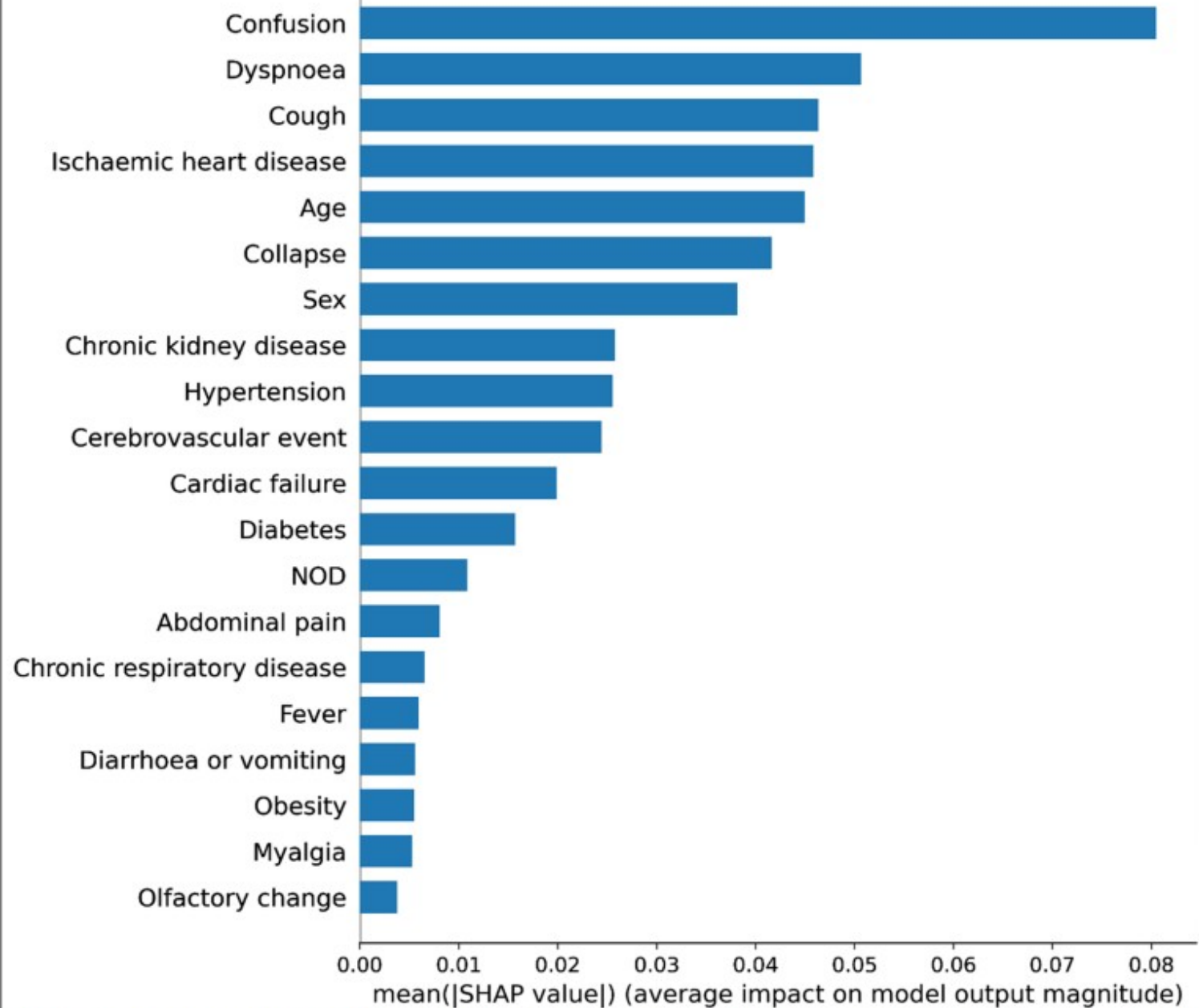


Fig. 4 Predictor importance as considered by an Artificial Neural Network trained and validated on 398 patients with SARS-CoV-2 in a West London hospital, during March 1–April 24, 2020. *SHAP value* Shapley additive explanations value. This approximates how much each predictor contributes to the average prediction for the dataset. *NOD* number of days of symptoms prior to hospital admission

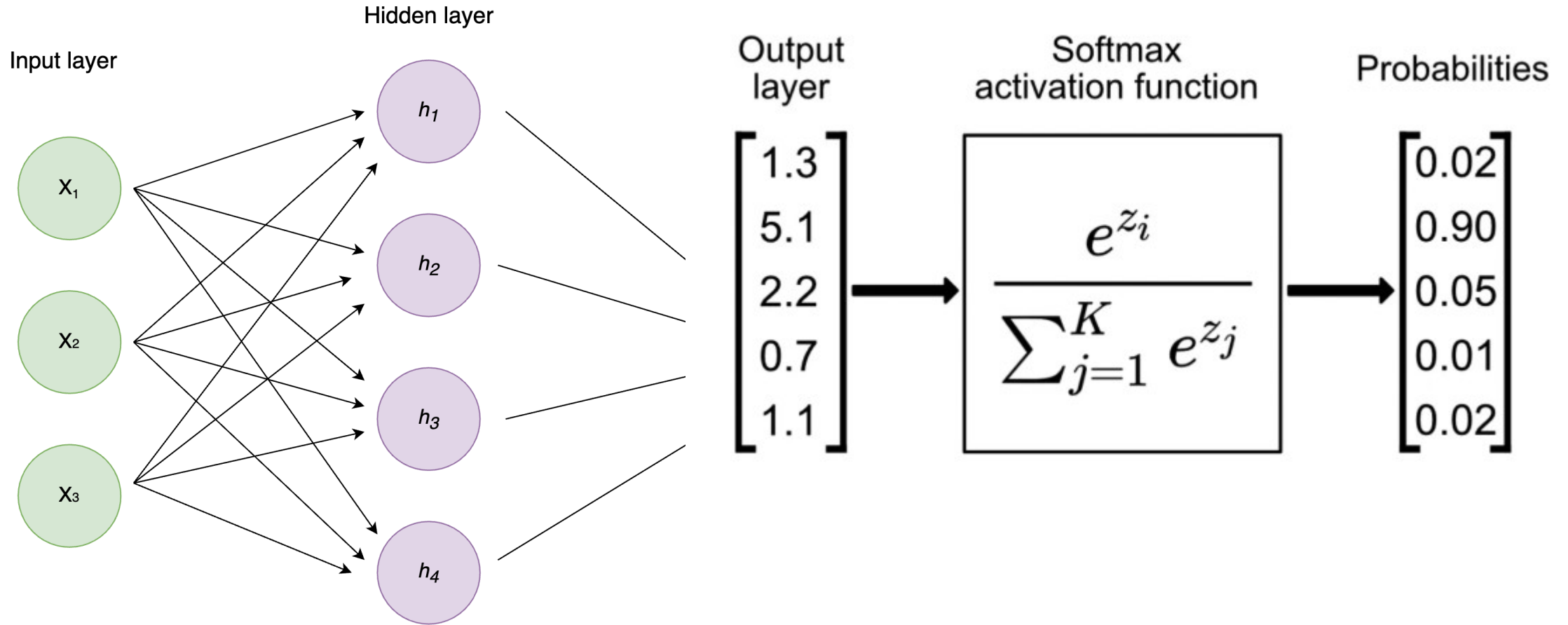
Multi-Class Classification

- What if we want to classify into more than two groups?
 - One vs all approach
 - One vs one approach
- Define a “one hot” output vector where the vector at a given index is closer to one if there’s a higher probability of it actually being in that class.

$$\mathbf{y} = [y_1, \dots, y_K]^T$$

$y_k = 1$ if the example is of class k and $y_k = 0$ otherwise.

Softmax Function



Multiclass Log Loss

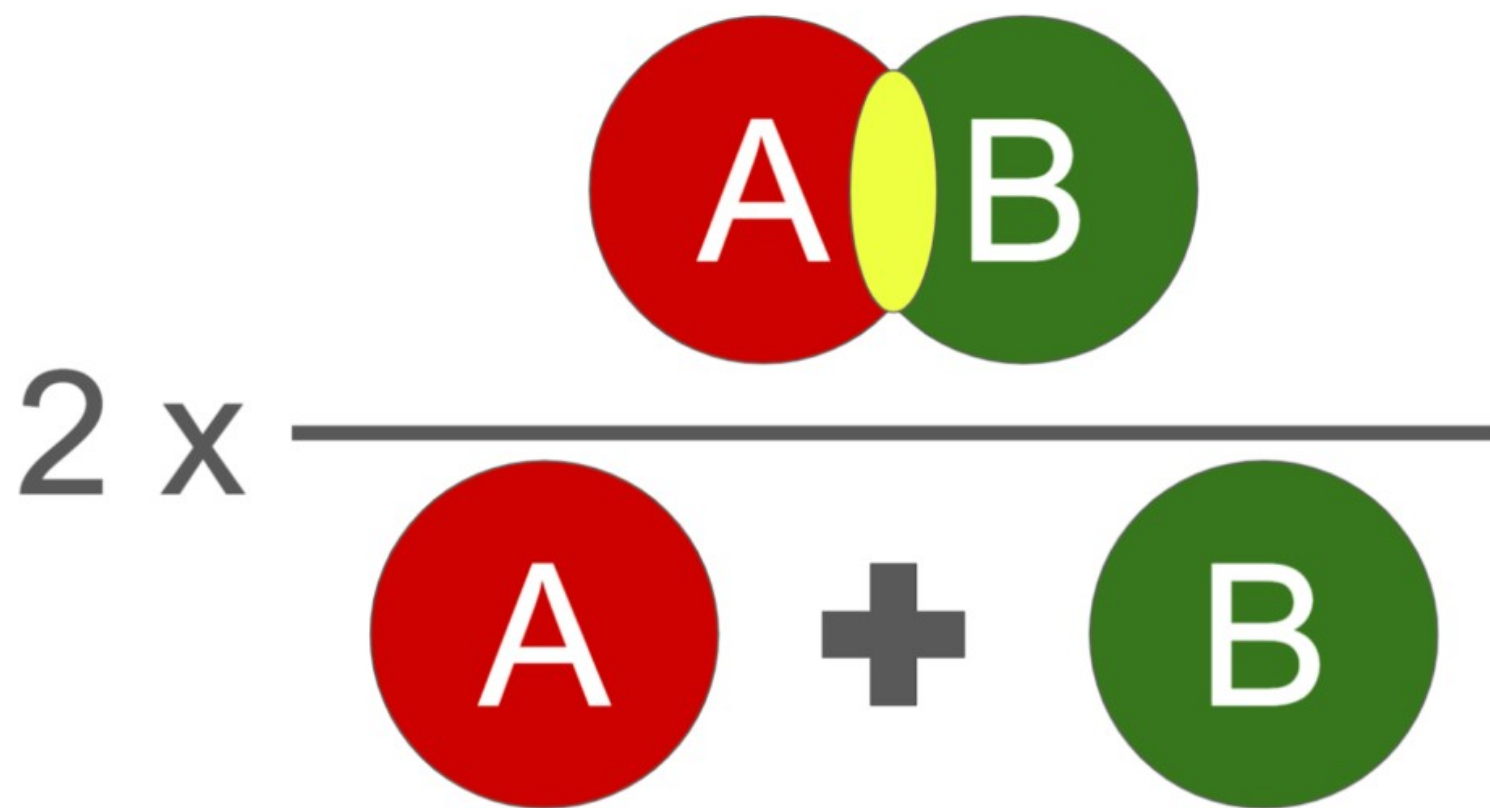
- For every class, multiply the log of the likelihood times if that is the true class.
- If matches, probability approaches 1 and log is 0.
- Extension of loss above.

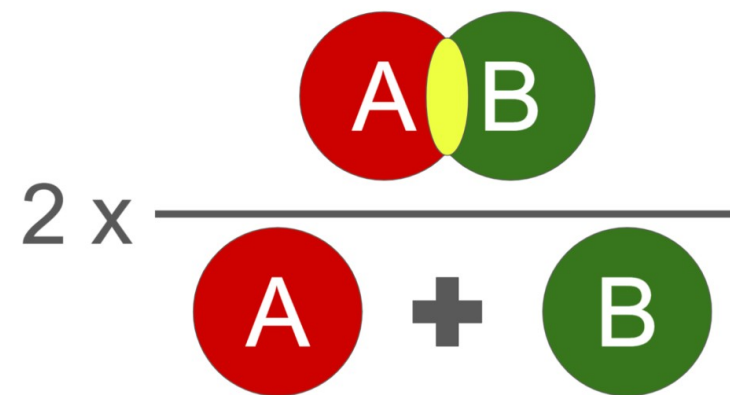
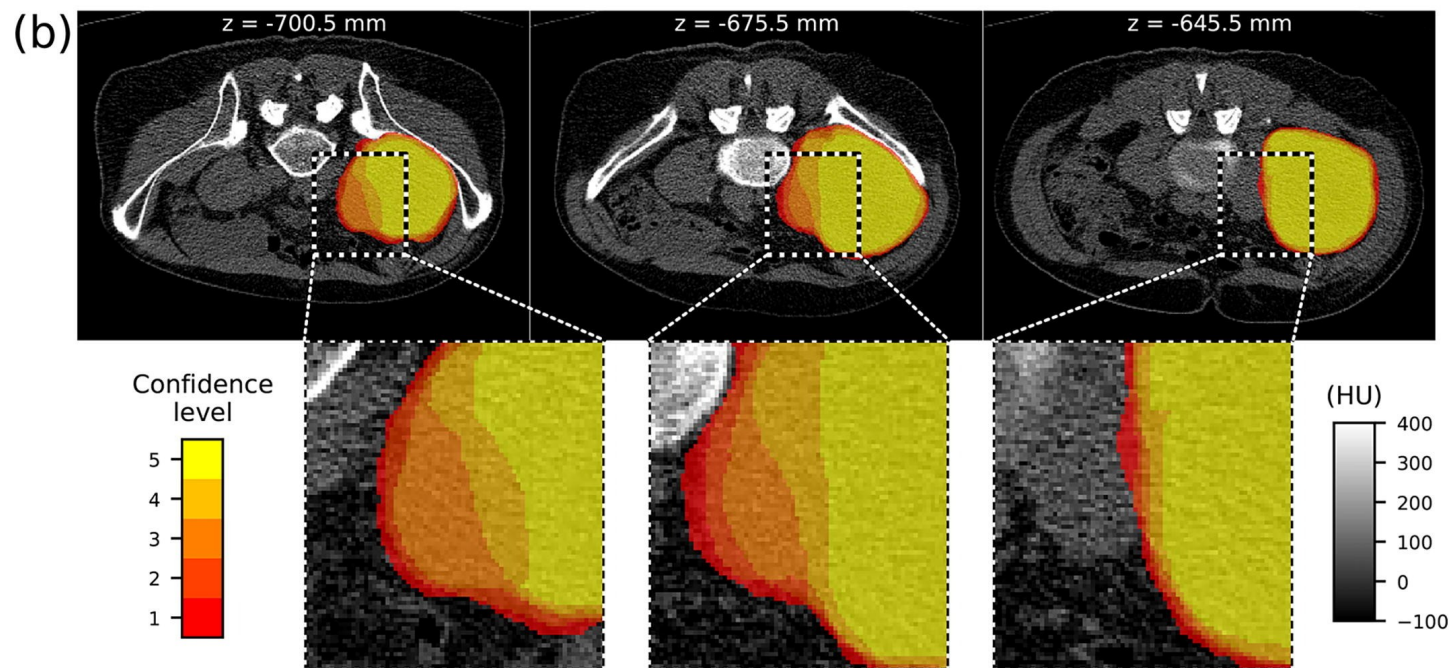
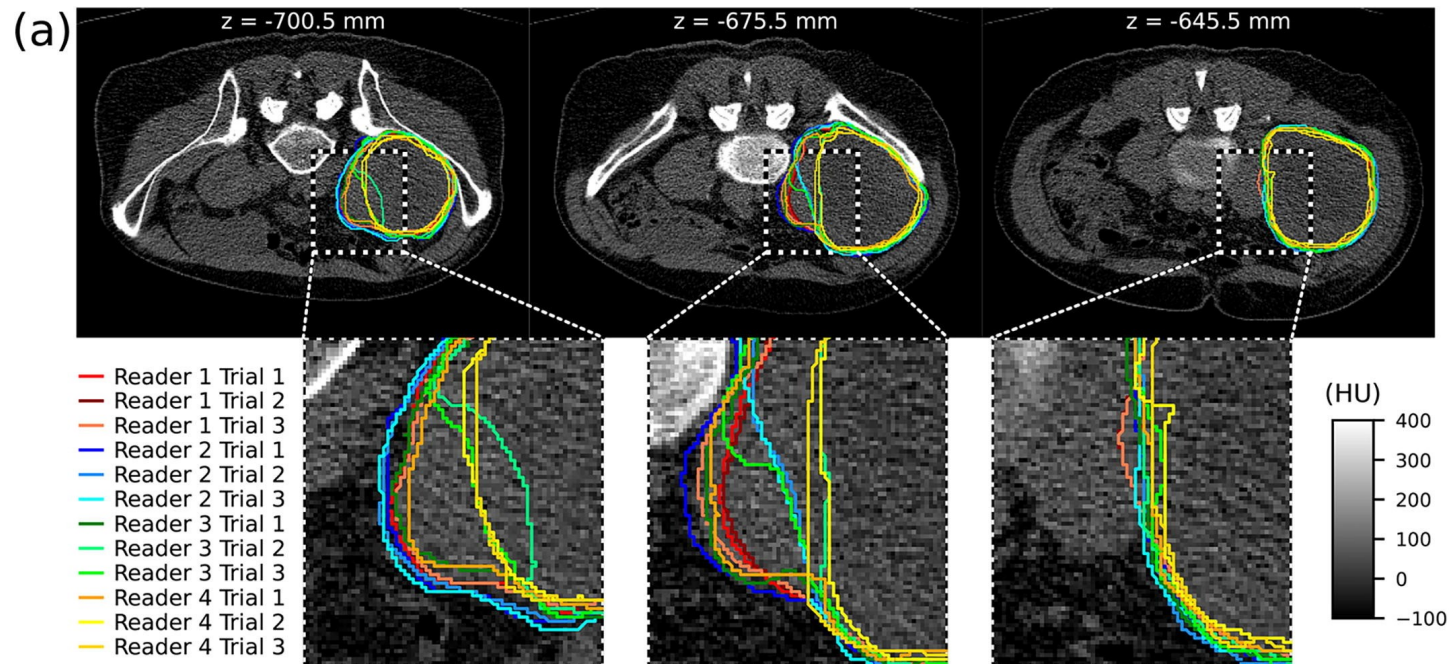
$$\mathcal{L}_{\text{nllm}}(\mathbf{g}, \mathbf{y}) = - \sum_{k=1}^K y_k \cdot \log(g_k)$$

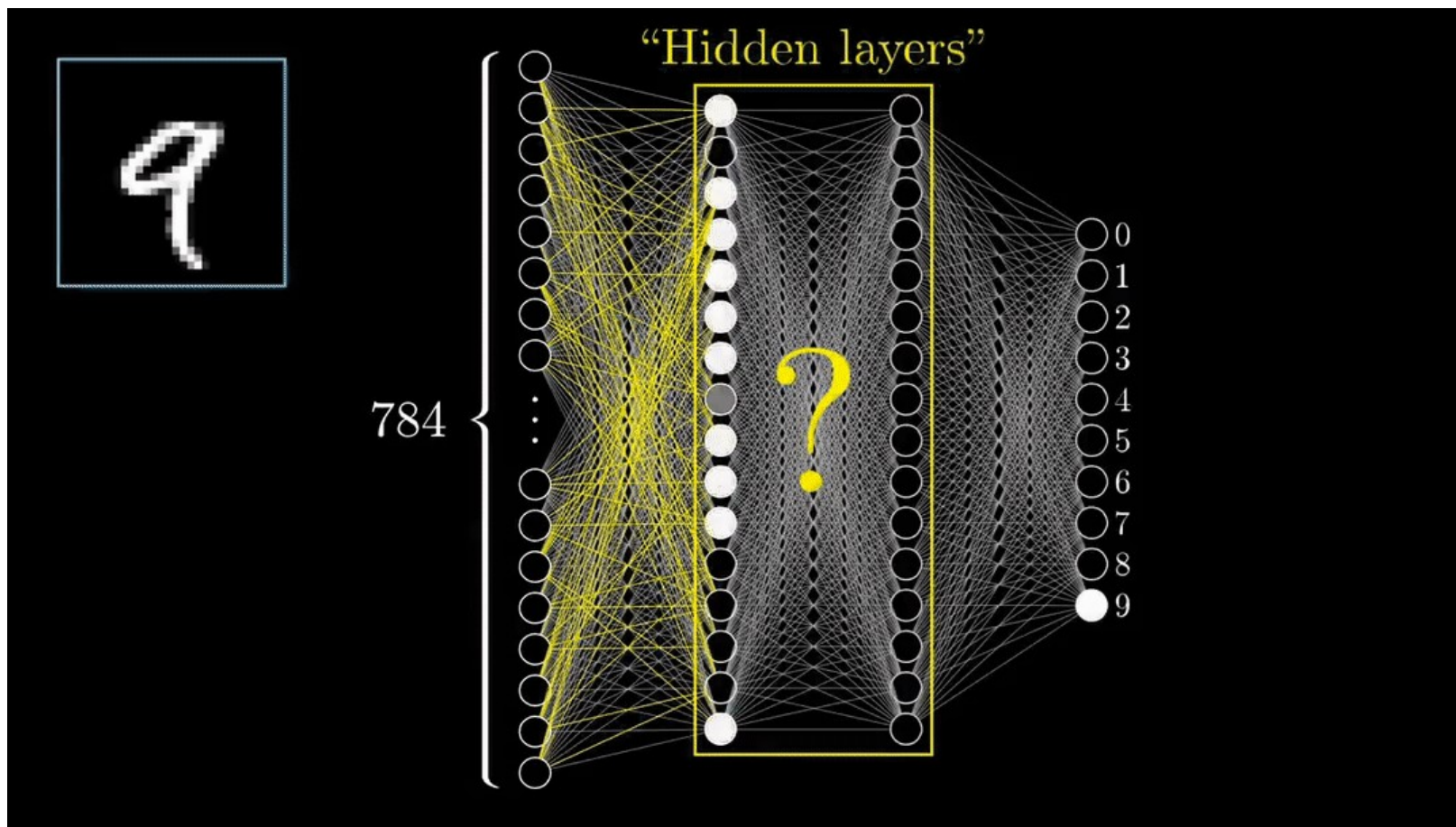
Recap

Problem Type	Final Activation	Loss Function
Predicting a number	Linear or ReLU	Mean Squared Error
Classifying into one of two groups	Sigmoid	Log Loss (Cross Entropy)
Classifying into one of many groups	Softmax	Multiclass Log Loss
Other		Many more!

Preview: DICE Loss

$$2 \times \frac{\text{Intersection of A and B}}{\text{Area of A} + \text{Area of B}}$$






MNIST Classification

Thanks!